

Universidade Federal de Lavras – UFLA

Bacharelado Interdisciplinar em Ciência e Tecnologia (BICT)

Projeto: Árvore Binária de Busca

GCT153 – Estruturas de Dados II

Discente(s):

Maria Julia Pedroso Pimenta
Juka Francis Pereira Gonçalves

Professor:

Prof. Dr. Alysson Naves

Sumário

1	Introdução	2
2	Estrutura da Implementação	2
2.1	Classe Nó	2
2.2	Classe ArvoreBinariaBusca	3
3	Operações Implementadas	3
4	Saídas dos Casos de Teste	4
5	Questões Teóricas	9
6	Conclusão	12
	Referências	13

1 Introdução

As árvores binárias de busca (do inglês *Binary Search Tree* BST) são estruturas de dados fundamentais em Ciência da Computação. Elas organizam informações de forma hierárquica, permitindo que operações de inserção, busca e remoção sejam realizadas, em média, em tempo logarítmico em relação ao número de elementos armazenados.

A propriedade central de uma BST é simples e elegante: para cada nó da árvore, todos os valores da subárvore esquerda são menores que o valor do nó, e todos os valores da subárvore direita são maiores. Essa invariante é o que garante eficiência nas operações, pois permite descartar metade do espaço de busca a cada comparação, comportamento análogo à busca binária em arrays ordenados.

Este relatório documenta o desenvolvimento de uma implementação completa de BST em Python, realizada como parte do Projeto final da disciplina GCT153 – Estruturas de Dados II, sob orientação do Prof. Dr. Alysson Naves. Foram implementadas todas as operações obrigatórias definidas no enunciado: inserção, busca, remoção (com tratamento dos três casos), percursos pré-ordem, em-ordem e pós-ordem, além de funções auxiliares para cálculo de altura, contagem de nós e folhas, verificação de árvore vazia e visualização da estrutura no terminal.

2 Estrutura da Implementação

A implementação foi dividida em dois arquivos principais:

- `bst.py`: contém as classes `No` e `ArvoreBinariaBusca` com todos os métodos fundamentais;
- `main.py`: interface interativa com menu de 13 opções que permite testar todas as operações em tempo real.

2.1 Classe Nó

Cada nó armazena três atributos: o valor, uma referência para o filho esquerdo e uma referência para o filho direito, ambas inicializadas como `None`.

2.2 Classe ArvoreBinariaBusca

A classe mantém apenas a referência para a raiz da árvore. Todos os métodos públicos (sem underscore) delegam a chamada para um método privado recursivo que recebe o nó atual como parâmetro, seguindo exatamente os pseudocódigos fornecidos no enunciado.

Conforme permitido pelas regras do enunciado, optou-se por ignorar silenciosamente valores repetidos. Quando um valor já presente na árvore é inserido novamente, o algoritmo simplesmente retorna o nó atual sem modificar nada. Essa decisão mantém a simplicidade da estrutura e garante que cada valor ocorra no máximo uma vez.

3 Operações Implementadas

Inserção

A inserção é feita de forma recursiva. Se o nó atual for None, cria-se um novo nó com o valor. Caso contrário, o valor é comparado com o do nó atual e a busca continua à esquerda (valor menor) ou à direita (valor maior).

Busca

A busca também é recursiva. Retorna True se o valor for encontrado e False se o percurso chegar a um nó None sem encontrá-lo.

Mínimo e Máximo

O menor valor está sempre no nó mais à esquerda da árvore (segue-se esq até que ele seja None). O maior, no nó mais à direita (segue-se dir).

Remoção

A remoção trata três casos distintos, 1. nó folha; 2. nó com apenas um filho; 3. nó com dois filhos. detalhados na Seção 5.

Percursos

Foram implementados os três percursos clássicos, cada um retornando uma lista com os valores na ordem de visita correspondente.

Funções de Informação

As funções `altura`, `contar_nos` e `contar_folhas` são implementadas de forma recursiva, seguindo os pseudocódigos da Seção 6 do enunciado. A convenção adotada para a altura é: árvore vazia = -1 ; árvore com apenas a raiz = 0 .

Visualização no Terminal

A função, `visualizar_arvore`, foi incluída para exibir a estrutura hierárquica da árvore diretamente no terminal, usando os caractere `- |` para representar as conexões entre pai e filhos. A visualização é acionada automaticamente após inserções e remoções e também pode ser chamada explicitamente pela opção 13 do menu.

4 Saídas dos Casos de Teste

A seguir são apresentadas a interface utilizada e as saídas geradas pela execução de sete casos de teste:

Interface

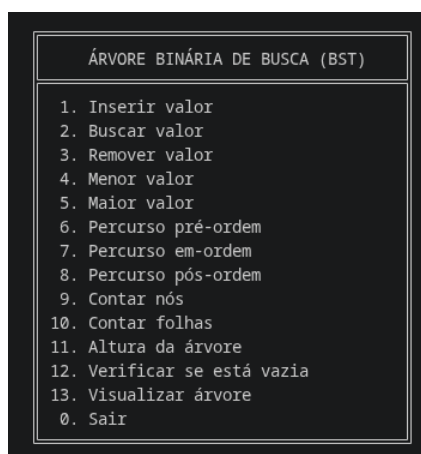


Figura 1: Interface para realizar operações na árvore.

Teste 1: Inserção Básica

Valores inseridos: 10, 5, 15, 2, 7, 12, 20

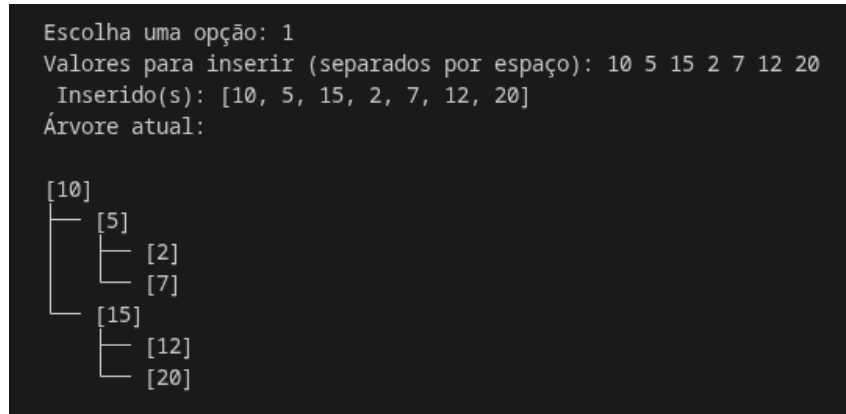


Figura 2: Inserção na árvore.

Teste 2: Busca

```

Escolha uma opção: 2
Valor a buscar: 7
7 encontrado na árvore.
  
```

Figura 3: Busca do número 7.

```

Escolha uma opção: 2
Valor a buscar: 12
12 encontrado na árvore.
  
```

Figura 4: Busca do número 12.

```

Escolha uma opção: 2
Valor a buscar: 99
99 não encontrado na árvore.
  
```

Figura 5: Busca do número 99.

```

Escolha uma opção: 2
Valor a buscar: 1
1 não encontrado na árvore.
  
```

Figura 6: Busca do número 1.

Teste 3: Menor e Maior Valor

```
Escolha uma opção: 4
Menor valor: 2
```

Figura 7: Menor valor da árvore.

```
Escolha uma opção: 5
Maior valor: 20
```

Figura 8: Maior valor da árvore

Teste 4: Remoção de Nó Folha

Removendo o valor 2 (nó sem filhos):

```
Escolha uma opção: 3
Valor a remover: 2
2 removido.
Árvore atual:
```

```
[10]
├── [5]
│   ├── (vazio)
│   └── [7]
└── [15]
    ├── [12]
    └── [20]
```

Figura 9: Remoção de nó folha.

```
Escolha uma opção: 7
Em-ordem: [5, 7, 10, 12, 15, 20]
```

Figura 10: Recurso em-ordem após remover nófolha.

Teste 5: Remoção de Nó com Um Filho

Valores inseridos: 10, 5, 15, 2, 7, 6. Removendo o nó 7 (que possui apenas o filho esquerdo 6):

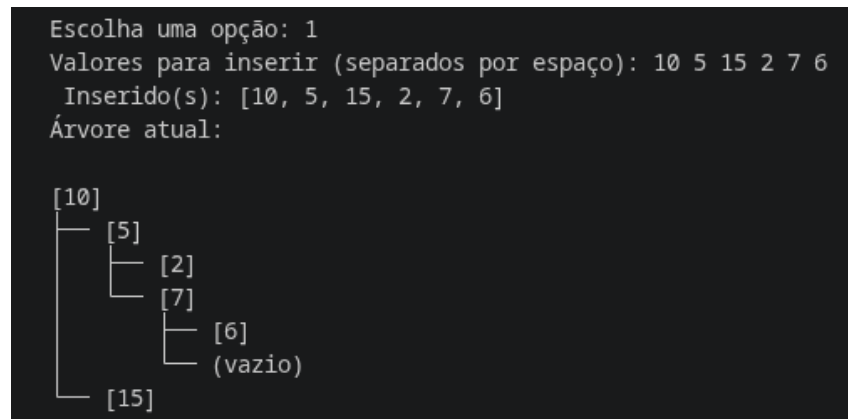


Figura 11: Inserção nova para remover nó com um filho.

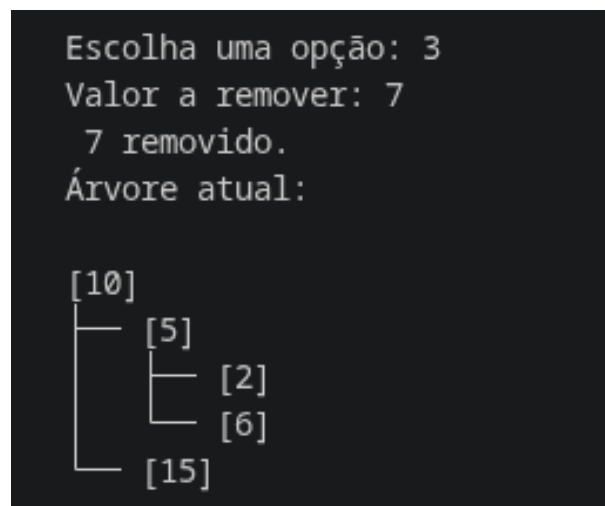


Figura 12: Remoção de nó com um filho.

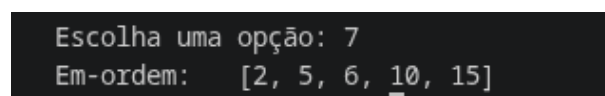


Figura 13: Recurso em-ordem após remover nó com um filho.

Teste 6: Remoção de Nó com Dois Filhos

Valores inseridos: 10, 5, 15, 2, 7, 12, 20. Removendo a raiz 10 (dois filhos). O sucessor em-ordem é 12 (menor da subárvore direita), que assume o lugar de 10:

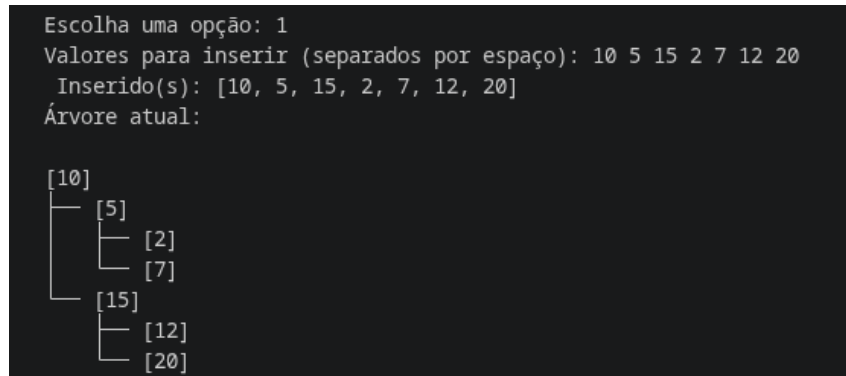


Figura 14: Inserção nova para remover nó com dois filhos.

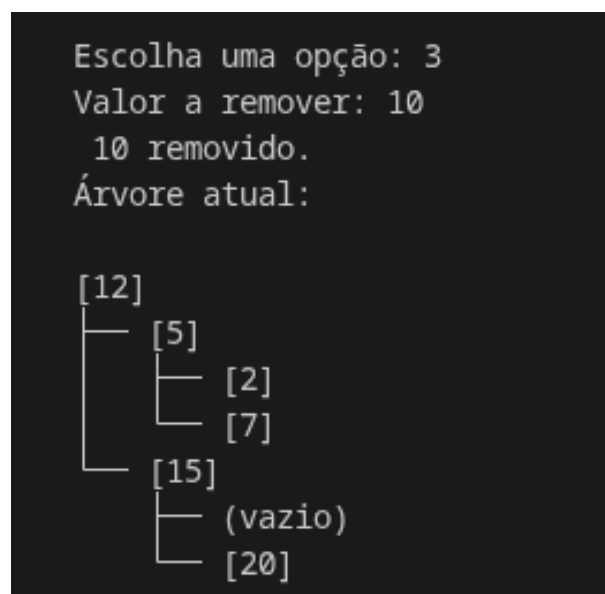


Figura 15: Remoção de nó com dois filho.

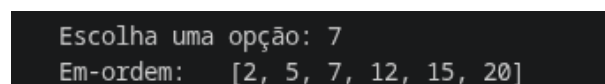


Figura 16: Recurso em-ordem após remover nó com dois filho.

Teste 7: Árvore Degenerada

Valores inseridos em ordem crescente: 1, 2, 3, 4, 5. Nenhum nó possui filho à esquerda; a árvore degenera em uma lista encadeada:

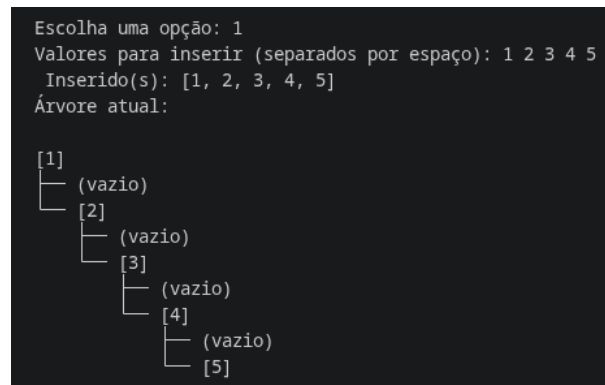


Figura 17: Árvore degenerada.

```

Escolha uma opção: 7
Em-ordem: [1, 2, 3, 4, 5]

```

Figura 18: Percurso em-ordem em árvore degenerada.

```

Escolha uma opção: 11
Altura da árvore: 4

```

Figura 19: Altura árvore degenerada.

```

Escolha uma opção: 9
Quantidade de nós: 5

```

Figura 20: Quantidade de nós em árvore degenerada.

```

Escolha uma opção: 10
Quantidade de folhas: 1

```

Figura 21: Quantidade de folhas em árvore degenerada.

5 Questões Teóricas

Por que o percurso em-ordem de uma BST gera os valores em ordem crescente?

O percurso em-ordem visita os nós na sequência: subárvore esquerda $\rightarrow$ nó atual $\rightarrow$ subárvore direita. Como a propriedade fundamental da BST garante que todos os valores à esquerda de qualquer nó são menores que ele, e todos à direita são maiores, essa ordem de visita naturalmente percorre os elementos do menor para o maior. Em outras palavras, o próprio invariante da BST é o que faz o em-ordem equivaler a uma ordenação crescente.

Qual a diferença entre uma árvore binária comum e uma árvore binária de busca?

Uma árvore binária é qualquer estrutura hierárquica em que cada nó possui no máximo dois filhos (esquerdo e direito), sem nenhuma restrição sobre os valores armazenados. A árvore binária de busca é um caso especial em que os valores obedecem a uma ordem: para todo nó n , os valores da subárvore esquerda são estritamente menores que $n.valor$ e os da subárvore direita são estritamente maiores. É justamente essa restrição que permite realizar buscas eficientes, pois cada comparação descarta metade da árvore.

Por que a altura da árvore influencia o desempenho?

As operações de busca, inserção e remoção percorrem um caminho da raiz até o nó de destino (ou até uma folha, no caso de inserção). O comprimento máximo desse caminho é exatamente a altura da árvore. Logo, o custo dessas operações é $O(h)$, onde h é a altura. Em uma BST balanceada com n nós, $h \approx \log_2 n$, o que é ótimo. Em uma árvore degenerada, $h = n - 1$ e o desempenho cai para $O(n)$, equivalente a uma lista encadeada.

O que acontece quando os valores são inseridos já ordenados?

Quando os valores chegam em ordem crescente (ou decrescente), cada novo

elemento é sempre maior (ou sempre menor) que todos os anteriores. Isso faz com que ele seja inserido sempre como filho direito (ou esquerdo) do último nó, sem ramificações. A árvore resultante é chamada de degenerada ou desbalanceada: ela tem formato de lista encadeada, com altura $h = n - 1$. O Teste 7 ilustra exatamente esse comportamento: ao inserir 1, 2, 3, 4, 5, a altura obtida foi 4 e havia apenas 1 folha.

Qual o custo médio esperado para busca em uma BST balanceada?

Em uma BST balanceada com n nós, a altura é $h = \lfloor \log_2 n \rfloor$. Como a busca percorre no máximo um caminho da raiz até uma folha, o custo médio e o pior caso são ambos $O(\log n)$. Para uma árvore com 10^6 nós, por exemplo, seriam necessárias no máximo cerca de 20 comparações.

Qual o pior caso para busca em uma BST degenerada?

Em uma BST degenerada (valores inseridos em ordem crescente ou decrescente), a altura é $n - 1$. Para buscar um valor que não existe ou que está na última posição, é necessário percorrer todos os n nós. O pior caso é, portanto, $O(n)$, o mesmo que uma busca linear em um array ou lista.

Explique os três casos de remoção

A remoção em uma BST deve preservar a propriedade de busca após a retirada do nó. Para isso, distinguem-se três situações:

1. **Nó folha (sem filhos):** O nó pode ser removido diretamente, basta fazer com que o pai aponte para None no lugar do filho removido. É o caso mais simples.
2. **Nó com apenas um filho:** O nó é substituído pelo seu único filho. O pai do nó removido passa a apontar diretamente para o filho sobrevi-

vente. A propriedade da BST é preservada porque os valores abaixo do filho já respeitavam a ordem em relação ao pai.

3. **Nó com dois filhos:** Não é possível substituir o nó por qualquer um dos filhos sem violar a propriedade da BST. A solução é encontrar o sucessor em-ordem, que é o menor valor da subárvore direita do nó a ser removido. Esse valor é copiado para o nó atual e, em seguida, o sucessor é removido da subárvore direita (onde, por ser o mínimo, ele terá no máximo um filho, o direito). Assim a propriedade da BST é mantida.

6 Conclusão

Este trabalho permitiu compreender, de forma prática, a estrutura e o funcionamento das Árvores Binárias de Busca. A implementação manual de cada operação tornou evidente como a propriedade fundamental da BST (menores à esquerda, maiores à direita) é o alicerce de toda a eficiência da estrutura.

A análise dos sete casos de teste evidenciou os pontos fortes e as limitações das BSTs: enquanto uma árvore balanceada oferece desempenho $O(\log n)$ em todas as operações, a inserção de valores já ordenados produz o pior cenário possível, uma árvore degenerada com desempenho linear $O(n)$.

Os três casos de remoção ilustram o cuidado necessário para manter a propriedade de busca após alterações na árvore, especialmente no caso de nós com dois filhos, onde o uso do sucessor em-ordem é a estratégia que garante a correção da estrutura.

Como trabalho futuro, seria interessante implementar uma BST autoequilibrada, como a Árvore AVL ou a Árvore Rubro-Negra, que garantem altura

$O(\log n)$ independentemente da ordem de inserção, eliminando o problema da degeneração.

Referências

- CORMEN, T. H. et al. *Introduction to Algorithms*. 4. ed. MIT Press, 2022.
- SEDGEWICK, R.; WAYNE, K. *Algorithms*. 4. ed. Addison-Wesley, 2011.
- NAVES, A. *GCT153 – Estruturas de Dados II: Projeto 3 – Árvore Binária de Busca*. UNIFAL-MG, 2026.